Sprint Planner Pro

Technical Documentation

Repository: github.com/Kunal-Mishra10/Sprint-Planner-Pro

Author: Kunal Mishra | Version: 1.0 | April 2026

Stack: TypeScript · React · Express 5 · PostgreSQL · OpenAI GPT

Table of Contents

1. **System Architecture**
   - 1.1 Overview & Monorepo Structure
   - 1.2 Technology Stack
   - 1.3 High-Level Architecture
2. **Frontend Implementation**
   - 2.1 React + Vite Application
   - 2.2 Pages & Navigation
   - 2.3 UI Design System
   - 2.4 State Management & API Hooks
3. **Backend Logic & Logic Layer**
   - 3.1 Express 5 API Server
   - 3.2 Route Definitions
   - 3.3 AI Integration Logic
   - 3.4 Business Logic Layer
4. **Database Design**
   - 4.1 PostgreSQL + Drizzle ORM
   - 4.2 Schema Tables
   - 4.3 Entity Relationship Overview
5. **Admin Dashboard Overview**
   - 5.1 Dashboard Features
   - 5.2 Admin API Endpoints
6. **Git Commit Progression**
   - 6.1 Commit History Summary
   - 6.2 Development Timeline

1. System Architecture

> *Sprint Planner Pro is an AI-powered product requirement and sprint planning tool built as a pnpm workspace monorepo. It separates concerns into discrete packages enabling independent versioning and deployment of each layer.*

1.1 Overview & Monorepo Structure

The repository is organized as a pnpm workspace monorepo with three top-level directories: artifacts/ (deployable applications), lib/ (shared libraries), and scripts/ (developer tooling). Node.js 24 and TypeScript 5.9 are used throughout.

Directory	Package	Purpose
artifacts/sprint-planner	@workspace/sprint-planner	React + Vite frontend SPA
artifacts/api-server	@workspace/api-server	Express 5 REST API backend
lib/db	@workspace/db	PostgreSQL schema + Drizzle ORM
lib/api-spec	@workspace/api-spec	OpenAPI spec + Orval codegen
lib/integrations-openai	@workspace/integrations-openai-ai-server	OpenAI AI server integration
scripts/	—	Build & developer tooling

Table 1.1 — Monorepo package layout

1.2 Technology Stack

Runtime	Node.js 24
Language	TypeScript 5.9
Package Mgr	pnpm workspaces
Frontend	React 18 + Vite
Backend	Express 5
Database	PostgreSQL + Drizzle ORM
Validation	Zod (v4) + drizzle-zod
API Codegen	Orval (from OpenAPI spec)
Build Tool	esbuild (CJS bundle)
AI Provider	OpenAI GPT via Replit AI Integrations
Formatter	Prettier 3.x

Table 1.2 — Technology stack

1.3 High-Level Architecture

The architecture follows a classic three-tier pattern enriched by an AI generation layer. The browser SPA communicates with the Express API over HTTP. The API layer handles business logic and delegates AI generation to OpenAI. All persistent state lives in PostgreSQL via Drizzle ORM.

Tier	Component	Technology	Notes
Presentation	Frontend SPA	React 18 + Vite	Port 19948, dark indigo theme
API	REST API Server	Express 5 + TypeScript	/api/* prefix, esbuild CJS bundle
AI	GPT Pipeline	OpenAI GPT (gpt-5.4)	Structured JSON generation via prompt
Data	PostgreSQL DB	Drizzle ORM + drizzle-zod	4 tables, type-safe queries

Figure 1.1 — Three-tier + AI architecture

2. Frontend Implementation

> *The frontend is a single-page React application bundled by Vite, running on port 19948. It features a deep indigo / slate dark theme and communicates with the backend through auto-generated API hooks produced by Orval from the OpenAPI specification.*

2.1 React + Vite Application

Package name	@workspace/sprint-planner
Build tool	Vite
Framework	React 18
Dev port	19948
API hooks	Auto-generated by Orval from OpenAPI spec
Styling theme	Deep indigo / slate dark UI

2.2 Pages & Navigation

The application is organized into four primary pages, each mapped to a distinct route:

Page	Route	Description
Home / Generator	/	Feature idea input form; triggers AI PRD generation
Results	/results/:id	Displays generated PRD, task list, and sprint plan
Features Library	/features	Browsable list of all stored feature ideas and their statuses
Admin Dashboard	/admin	System statistics, recent activity feed, health metrics

Table 2.1 — Application pages

2.3 UI Design System

The UI follows a consistent dark-mode design language. Key design decisions include:

- **Color palette:** Deep indigo (#3730a3 – #1e1b4b) as primary, slate (#0f172a) for backgrounds, cyan (#06b6d4) for interactive accents.
- **Typography:** Arial / system sans-serif stack with bold weights for headings and medium for body text.
- **Component patterns:** Card-based layouts with subtle borders, status badges for feature lifecycle states (pending / generating / completed / failed), progress indicators for async AI generation.
- **Responsiveness:** Grid and flexbox layouts adapting from mobile to widescreen viewports.

2.4 State Management & API Hooks

All server state is managed through Orval-generated React Query hooks. These hooks are regenerated from the OpenAPI spec via:

```
pnpm --filter @workspace/api-spec run codegen
```

This ensures frontend and backend contracts remain in sync at compile time. Local UI state (form inputs, tab selection, loading overlays) is handled with React's built-in useState and useReducer hooks.

3. Backend Logic & Logic Layer

3.1 Express 5 API Server

The API server is an Express 5 application located at artifacts/api-server. It is bundled by esbuild into a CommonJS bundle and served at the /api path prefix.

Package	@workspace/api-server
Framework	Express 5
Bundle format	CJS (esbuild)
Validation	Zod v4 + drizzle-zod generated schemas
ORM	Drizzle ORM (PostgreSQL dialect)
AI client	@workspace/integrations-openai-ai-server

3.2 Route Definitions

The API exposes the following resource groups, each handling CRUD and specialized operations:

Route	Methods	Description
/api/features	GET, POST	List all features; create a new feature idea
/api/features/:id	GET, PATCH, DELETE	Read / update status / delete a feature
/api/prds	GET	List all generated PRDs
/api/prds/:featureId	GET	Retrieve PRD for a specific feature
/api/tasks	GET	List all tasks, optionally filtered by feature
/api/tasks/:id	GET, PATCH	Read or update a task
/api/sprints	GET	List sprint groupings with effort totals
/api/sprints/:id	GET	Get a specific sprint and its tasks
/api/admin/stats	GET	Aggregate counts and metrics for dashboard
/api/admin/recent-activity	GET	Latest feature generation events feed
/api/ai/generate-prd	POST	Trigger AI generation pipeline for a feature

Table 3.1 — API route definitions

3.3 AI Integration Logic

The AI generation pipeline is the core value-add of the application. When POST /api/ai/generate-prd is called, the following sequence executes:

- **Step 1 — Feature lookup:** Load the feature record from PostgreSQL; confirm pending status.
- **Step 2 — Status update:** Atomically set feature status to generating so the UI displays a progress indicator.
- **Step 3 — Prompt construction:** Assemble a comprehensive prompt containing feature title, description, and instructions to produce structured JSON (PRD sections, tasks, sprint assignments).
- **Step 4 — GPT call:** Send prompt to OpenAI GPT (model: gpt-5.4) via @workspace/integrations-openai-ai-server.
- **Step 5 — Response parsing:** Validate the AI JSON response against a Zod schema before any database writes.
- **Step 6 — Priority & risk scoring:** Smart scoring logic computes a priorityScore (effort, business value, dependencies) and riskScore (complexity, dependency depth) per task.
- **Step 7 — Dependency detection:** Cross-reference task descriptions to identify and store dependency relationships.
- **Step 8 — Persistence:** Write PRD, tasks, and sprint assignments to PostgreSQL in a single transaction; update feature status to completed.
- **Step 9 — Error handling:** On any failure, set feature status to failed and return a structured error.

3.4 Business Logic Layer

Beyond the AI pipeline, the business logic layer handles:

- **Validation middleware:** Every request body is parsed through a Zod schema. Invalid payloads receive a 422 with field-level error messages.
- **Status machine:** Features follow a strict state machine: pending → generating → completed | failed. Transitions are enforced at the service layer.
- **Effort aggregation:** Sprint totals are computed by summing task effortPoints within each sprint, exposed on /api/sprints.
- **Dependency graph:** Task dependencies are stored as foreign-key references and can be traversed for topological ordering.

4. Database Design

> *The database layer uses PostgreSQL accessed through Drizzle ORM with full TypeScript type safety. Schema definitions live in lib/db/schema.ts and are shared across all packages. Schema changes are applied via drizzle-kit push (dev) or drizzle-kit migrate (prod).*

4.1 PostgreSQL + Drizzle ORM

Database	PostgreSQL (latest)
ORM	Drizzle ORM
Type safety	drizzle-zod for auto-generated Zod insert/select schemas
Migrations	drizzle-kit push (dev) / drizzle-kit migrate (prod)
Package	@workspace/db (lib/db)
Connection	Environment variable DATABASE_URL

4.2 Schema Tables

Table: features

Column	Type	Constraints	Description
id	serial	PRIMARY KEY	Auto-incrementing identifier
title	text	NOT NULL	Short feature idea title
description	text	NOT NULL	Detailed feature description
status	text	NOT NULL, DEFAULT pending	pending | generating | completed | failed
createdAt	timestamp	NOT NULL, DEFAULT now()	Record creation timestamp
updatedAt	timestamp	NOT NULL, DEFAULT now()	Last modification timestamp

Table: prds

Column	Type	Constraints	Description
id	serial	PRIMARY KEY	Auto-incrementing identifier
featureId	integer	FK → features.id	Owning feature
overview	text	NOT NULL	High-level product overview
goals	text[]	NOT NULL	Array of product goals
userStories	jsonb	NOT NULL	Structured user story objects
techRequirements	text[]	NOT NULL	Technical requirement list
successMetrics	text[]	NOT NULL	KPIs and success criteria
createdAt	timestamp	NOT NULL	Creation timestamp

Table: tasks

Column	Type	Constraints	Description
id	serial	PRIMARY KEY	Auto-incrementing identifier
featureId	integer	FK → features.id	Parent feature
sprintId	integer	FK → sprints.id	Assigned sprint
title	text	NOT NULL	Task title
description	text	NOT NULL	Detailed task description
type	text	NOT NULL	feature | bug | chore | spike
priority	text	NOT NULL	low | medium | high | critical
effortPoints	integer	NOT NULL	Story point estimate
riskLevel	text	NOT NULL	low | medium | high
priorityScore	numeric		Computed priority score (0–100)
riskScore	numeric		Computed risk score (0–100)
dependencies	integer[]		Array of dependent task IDs
status	text	DEFAULT todo	todo | in-progress | done

Table: sprints

Column	Type	Constraints	Description
id	serial	PRIMARY KEY	Auto-incrementing identifier
featureId	integer	FK → features.id	Parent feature
sprintNumber	integer	NOT NULL	Ordinal sprint number (1, 2, 3…)
goal	text	NOT NULL	Sprint goal statement
totalEffort	integer	NOT NULL	Sum of effortPoints in sprint
createdAt	timestamp	NOT NULL	Creation timestamp

Tables 4.1–4.4 — Full schema definition

4.3 Entity Relationship Overview

The four tables form a parent-child hierarchy rooted at features. One feature produces exactly one PRD, one or more sprints, and multiple tasks. Tasks reference their sprint for grouping and may reference other tasks via the dependencies array column.

Relationship	Cardinality	Description
features → prds	1 : 1	Each feature generates exactly one PRD document
features → sprints	1 : N	One feature maps to multiple ordered sprints
features → tasks	1 : N	One feature produces many granular tasks
sprints → tasks	1 : N	Tasks are grouped within a sprint
tasks → tasks	M : N (self)	Tasks may depend on other tasks (dependency array)

Table 4.5 — Entity relationships

5. Admin Dashboard Overview

> *The Admin Dashboard provides operators with a real-time view of system health, generation throughput, and recent activity. It is accessible at /admin in the frontend and is powered by two dedicated API endpoints.*

5.1 Dashboard Features

- **Aggregate statistics:** Total features, PRDs generated, tasks created, sprints planned, and success/failure rates displayed as metric cards.
- **Status breakdown:** Counts of features in each lifecycle state (pending, generating, completed, failed) shown as a mini status chart.
- **Recent activity feed:** Chronological log of the most recent AI generation events, including feature title, status, timestamps, and duration.
- **Effort distribution:** Total effort points allocated across sprints, useful for capacity planning.
- **Risk overview:** Counts of high/medium/low risk tasks across the entire backlog, surfacing items that warrant additional review.
- **Live refresh:** The dashboard polls admin endpoints on a configurable interval, ensuring metrics stay current during active generation sessions.

5.2 Admin API Endpoints

Endpoint	Response Fields	Description
/api/admin/stats	totalFeatures, totalPrds, totalTasks, totalSprints, statusCounts, avgEffort	Aggregated system-wide metrics for the stats cards
/api/admin/recent-activity	id, featureTitle, status, createdAt, completedAt, duration	Paginated list of recent generation events ordered by recency

Table 5.1 — Admin API endpoints

Both endpoints require no authentication in the current implementation (suitable for Replit-hosted internal deployments). For production use, Bearer token or session middleware should be added in front of the /api/admin/* route group.

6. Git Commit Progression

> *The repository contains 14 commits on the main branch (KunalMishra2), spanning from April 18 to April 26, 2026. The history reflects a rapid, iterative build — from an initial Replit agent scaffold through core feature delivery, UI refinements, and documentation — all within a single week.*

6.1 Commit History (Chronological — Oldest to Newest)

#	Commit Hash	Date	Message	Description
1	8337c4b	Apr 18, 2026	Initial commit	Project bootstrapped by replit-agent. Sets up the monorepo scaffold: pnpm workspace, tsconfig, .replit, base package structure.
2	dcdacce	Apr 22, 2026	Add core functionality for generating sprint plans from feature ideas	Core AI generation pipeline implemented. POST /api/ai/generate-prd wired up; PRD, task, and sprint records persisted to PostgreSQL after GPT call.
3	fc6c308	Apr 22, 2026	Update application name to Sprint Planner and improve sidebar subtitle	Branding update. App renamed to 'Sprint Planner'; sidebar subtitle copy improved for clarity.
4	24fb9b9	Apr 22, 2026	Add feature owner tracking and reporting capabilities to the sprint planner	Feature Owner field introduced to the features table and UI. Reporting view updated to surface owner attribution across generated plans.
5	8c966f1	Apr 24, 2026	Make the Feature Owner field a mandatory requirement	Validation tightened: Feature Owner is now required at the schema and UI level. Existing Zod schema updated; form enforces the field before submission.
6	a29fb75	Apr 24, 2026	Add task counts above bars in admin dashboard	Admin dashboard UX improvement. Numeric task counts displayed above each bar in the effort/status charts for quicker at-a-glance reading.
7	2f5c62a	Apr 24, 2026	Published your App	Replit publish event — application deployed and made publicly accessible via Replit hosting for the first time.
8	261cec4	Apr 24, 2026	Update the application name from Sprint Planner to Sprint Generator	Second rename. Application title changed from 'Sprint Planner' to 'Sprint Generator' across UI and metadata.
9	c960e3f	Apr 24, 2026	Published your App	Second Replit publish event following the rename. Ensures the Sprint Generator branding is live in the deployed environment.
10	7cbc8d5	Apr 25, 2026	Rename the application to Sprint Planner	Final rename reverts back to 'Sprint Planner' as the canonical product name, consolidating the branding after earlier iterations.

#	Commit Hash	Date	Message	Description
11	9138818	Apr 25, 2026	Published your App	Third publish event — Sprint Planner name change deployed to production.
12	3488519	Apr 26, 2026	Transitioned from Plan to Build mode	Replit workflow mode switch from planning/scaffolding to active build mode. Signals shift to feature-complete development phase.
13	a29201a	Apr 26, 2026	Generate PDF technical documentation for Sprint Planner	Automated PDF documentation generation added. Produces a technical reference doc (architecture, routes, schema) from the codebase.
14	7367d37	Apr 26, 2026	Update documentation and scripts to reflect current project status	Documentation and helper scripts updated to align with the current state of the codebase, post-rename and post-feature additions.
15	2e97ee9	Apr 26, 2026	feat: add PDF download for sprint plans on Results page	User-facing feature: Results page now offers a PDF export button, allowing users to download their generated sprint plan as a PDF document.

Table 6.1 — Full commit history (15 commits, branch: main, author: KunalMishra2)

6.2 Development Timeline & Phases

The commit history maps cleanly to four development phases across a single week, demonstrating a rapid AI-assisted build cycle typical of Replit-hosted projects:

Phase	Date(s)	Commits	Key Deliverables
Foundation	Apr 18	#1	Replit agent initial scaffold — monorepo, workspace config, base structure
Core Feature Build	Apr 22	#2 – #4	AI sprint generation pipeline, Feature Owner tracking, app rename to Sprint Planner
UI & Admin Polish	Apr 24 – 25	#5 – #11	Mandatory owner field, admin chart labels, three publish events, Sprint Generator rename and revert
Documentation & Export	Apr 26	#12 – #15	Build mode transition, PDF technical docs generated, scripts updated, PDF download on Results page

Table 6.2 — Development phases

Notable observations from the commit log:

- **Rapid iteration:** The entire application was built and deployed within 9 days (Apr 18 to Apr 26, 2026).
- **Three publish events (commits #7, #9, #11):** Indicate the app was iteratively deployed to Replit hosting throughout development rather than in a single release.
- **Name instability (commits #3, #8, #10):** The app cycled through 'Sprint Planner' -> 'Sprint Generator' -> 'Sprint Planner', settling on Sprint Planner as the canonical name.
- **Documentation-first close (commits #13, #14):** The final sprint includes auto-generated PDF documentation and script updates, reflecting a mature hand-off mindset.
- **User-facing PDF export (commit #15):** The most recent commit adds a downloadable sprint plan PDF on the Results page — a high-value end-user feature.

Total commits: 15 | **Branch:** main | **Author:** KunalMishra2 | **Span:** Apr 18 – Apr 26, 2026 (9 days)